

FUNDACIÓN FULGOR
Communication Protocols

Practical Work 3 Report

Ethernet physical-layer labs: MAC, XGMII, 64b/66b PCS and MLD with
hardware-in-the-loop simulation

Student: Francisco Javier Vasquez Curetti

Instructors: Facundo Sposetti, Jorge Manuel Finochietto

September 2026

Contents

1	Introduction	1
2	Working environment	1
3	Lab 1: raw Ethernet frame (8-bit bus)	2
3.1	Goal	2
3.2	Design	2
3.3	Results	2
4	Lab 2: 64-bit XGMII interface	3
4.1	Goal	3
4.2	Results	4
5	Lab 3: 64b/66b PCS and scrambler	5
5.1	Goal	5
5.2	Design	5
5.3	Results: 64b/66b encoding	6
5.4	Results: scrambler	7
6	Lab 4: 100GBASE-R PCS and Multi-Lane Distribution (MLD)	8
6.1	Goal	8
6.2	Real mechanism (clause 82) and the lab abstraction	8
6.3	Execution	9
6.4	Answers to the assignment	9
7	General conclusions	10

Two conventions help read every trace:

- **Time scale.** The wrapper advances simulation time by one unit per half period and the trace uses `timescale 1ps`, so in GTKWave **one clock cycle equals 2 ps**. These are nominal times, not physical ones.
- **Byte order.** On the 64-bit buses, lane 0 (the first byte transmitted) sits in bits [7:0]. GTKWave displays the value in hexadecimal with the most significant bit on the left, so each word reads “backwards”: from right to left, one byte at a time.

3 Lab 1: raw Ethernet frame (8-bit bus)

3.1 Goal

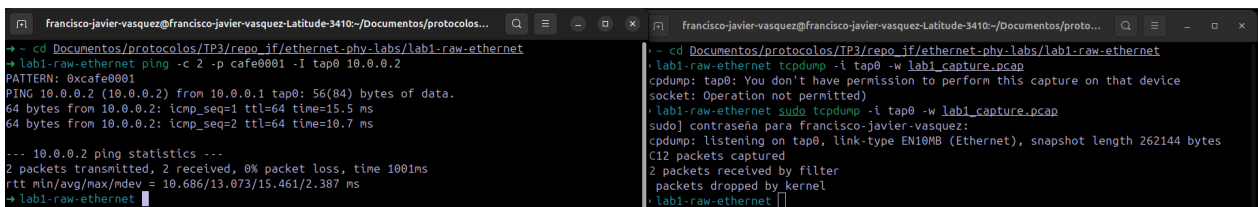
Set up the basic HIL pipeline and recognize the anatomy of an Ethernet II frame as the kernel hands it to a TAP interface: destination address (6 B), source address (6 B), EtherType (2 B) and payload. At this level **there is no preamble, SFD or FCS**: in real hardware the MAC/PHY adds and strips them. The goal is to find those same bytes, one per cycle, in the trace.

3.2 Design

`top.sv` is minimal: two independent registers ($A \rightarrow B$ and $B \rightarrow A$) carrying 8 data bits plus a *valid* bit, with no framing logic at all. Wireshark does all of the frame interpretation; the hardware only carries bytes. The actual ports are `a_data_in/a_valid_in`, `b_data_out/b_valid_out` (and their mirror images), not the `eth_txd/eth_tx_en` named in the assignment.

3.3 Results

`ping -c 2 -p cafe0001` got 2 out of 2 replies (0 % loss) and `tcpdump` recorded 12 packets on `tap0` (figure 1). Besides the two *Echo Request/Echo Reply* pairs, there are ARP messages in both directions and system traffic (ICMPv6 *Router Solicitation* and mDNS), because the version of the network script used in this lab did not yet disable IPv6.



```

francisco-javier-vasquez@francisco-javier-vasquez-Latitude-3410:~/Documentos/protocolos...
→ cd Documentos/protocolos/TP3/repo_if/ethernet-phy-labs/lab1-raw-ethernet
→ lab1-raw-ethernet ping -c 2 -p cafe0001 -I tap0 10.0.0.2
PATTERN: 0xcafe0001
PING 10.0.0.2 (10.0.0.2) from 10.0.0.1 tap0: 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=15.5 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=10.7 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 10.686/13.073/15.461/2.387 ms
→ lab1-raw-ethernet

francisco-javier-vasquez@francisco-javier-vasquez-Latitude-3410:~/Documentos/protocolos...
→ cd Documentos/protocolos/TP3/repo_if/ethernet-phy-labs/lab1-raw-ethernet
→ lab1-raw-ethernet tcpdump -i tap0 -w lab1_capture.pcap
tcpdump: tap0: You don't have permission to perform this capture on that device
socket: Operation not permitted
→ lab1-raw-ethernet sudo tcpdump -i tap0 -w lab1_capture.pcap
sudo! contraseña para francisco-javier-vasquez:
tcpdump: listening on tap0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
C12 packets captured
2 packets received by filter
0 packets dropped by kernel
→ lab1-raw-ethernet

```

Figure 1: Lab 1: ping with the `cafe0001` pattern (left) and `tcpdump` capture on `tap0` (right).

In Wireshark (figure 2), the *Echo Request* is 98 bytes long: destination `b6:cb:dc:76:cc:85` (`tap1`), source `02:ca:ee:44:4a:fa` (`tap0`), EtherType `0x0800` (IPv4), and ICMP data with `ca fe 00 01` repeated. The ARP frames are 42 bytes long, below the 60-byte Ethernet minimum, precisely because there is no padding or FCS at the TAP level. In the *Echo Reply* the addresses are swapped.

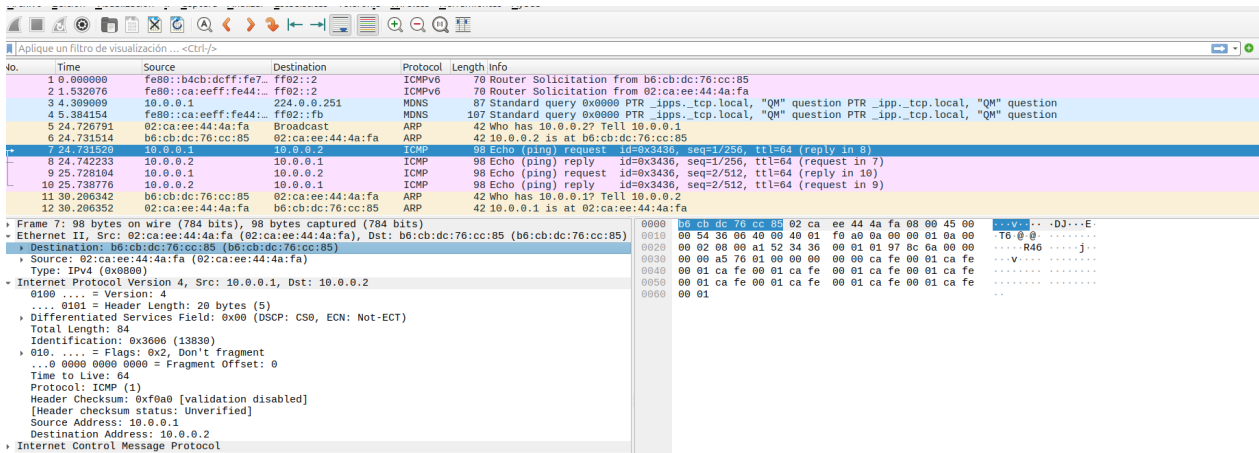


Figure 2: Lab 1: *Echo Request* in Wireshark: MAC addresses, EtherType 0x0800 and the ca fe 00 01 pattern in the hex dump.

In GTKWave (figure 3), while `a_valid_in = 1`, the `a_data_in[7:0]` bus goes through B6 CB DC 76 CC 85 | 02 CA EE 44 4A FA | 08 00 | 45 00 ...: exactly the bytes of the Wireshark hex dump, in the same order, one per cycle. The CA FE 00 01 pattern shows up later. A 98-byte frame takes 98 bus cycles. When the *Request* ends (`a_valid_in` goes back to 0), the *Reply* comes in on `b_data_in` with destination and source swapped (02 CA EE 44 4A FA B6 CB DC 76 CC 85 08 00 ...).

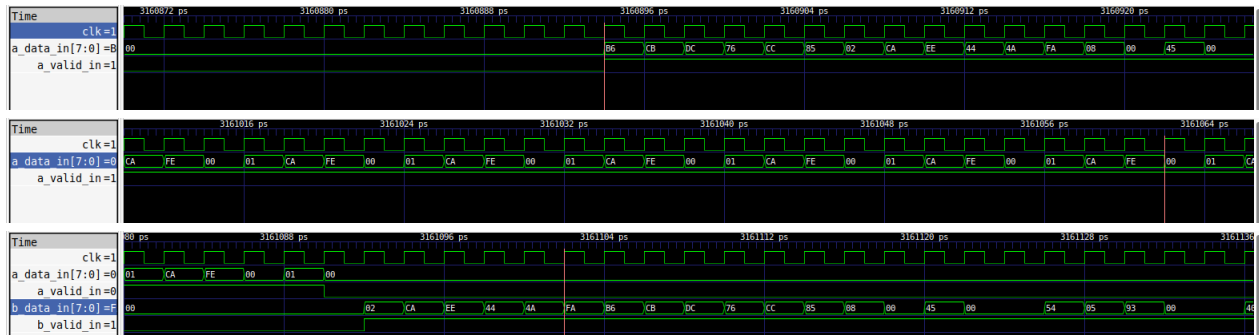


Figure 3: Lab 1: `a_data_in` in GTKWave. Top: start of the frame (destination MAC, source MAC, EtherType, start of IPv4). Middle: ICMP data CA FE 00 01. Bottom: end of the *Request* and start of the *Reply* on `b_data_in`.

Latency. The RTL adds a single register per direction. However, in the trace `b_data_out` takes each byte on the same edge at which it appears on `a_data_in` (checked on the VCD). The reason is that the wrapper updates the input and applies the rising edge in the same call to `eval()`, so the register captures the new value on that very edge and the trace shows no offset between input and output.

Conclusion. The frame shown by Wireshark is exactly the byte sequence that goes through the RTL. This lab sets the method used in the rest of the assignment: locating in the trace, byte by byte, what is seen in the capture.

4 Lab 2: 64-bit XGMII interface

4.1 Goal

Move from one byte per cycle to the 10G XGMII interface (IEEE 802.3, clause 46): 8 lanes of 8 bits (TXD[63:0]) plus one control bit per lane (TXC[7:0], 1 = control character). Control characters

delimit the frame: $/I/ = 0x07$ (idle), $/S/ = 0xFB$ (start, always on lane 0) and $/T/ = 0xFD$ (terminate, on any lane). $/S/$ is followed by the preamble (0x55) and the SFD (0xD5), and the frame ends with the FCS (CRC-32).

Version used. The lab was done with the version of the repository available at the time (lab2-xgmii-64bit). The next day the course replaced it with lab2-cgmii, which implements the MAC in RTL. In the version used, `wrapper.cpp` builds the XGMII framing: it assembles the words, computes and appends the FCS, and checks it on reception. `top.sv` registers the XGMII words (one stage per direction) and extracts header fields for debugging. The analysis of the XGMII bus is the same in both versions.

4.2 Results

ping -c 2 -p cafe0001 again got 2 out of 2 replies and tcpdump recorded 8 packets (mDNS, ARP, two *Echo* pairs and ICMPv6). Figure 4 shows that the *Echo Request* is identical to the one in lab 1: the XGMII interface does not modify the frame.

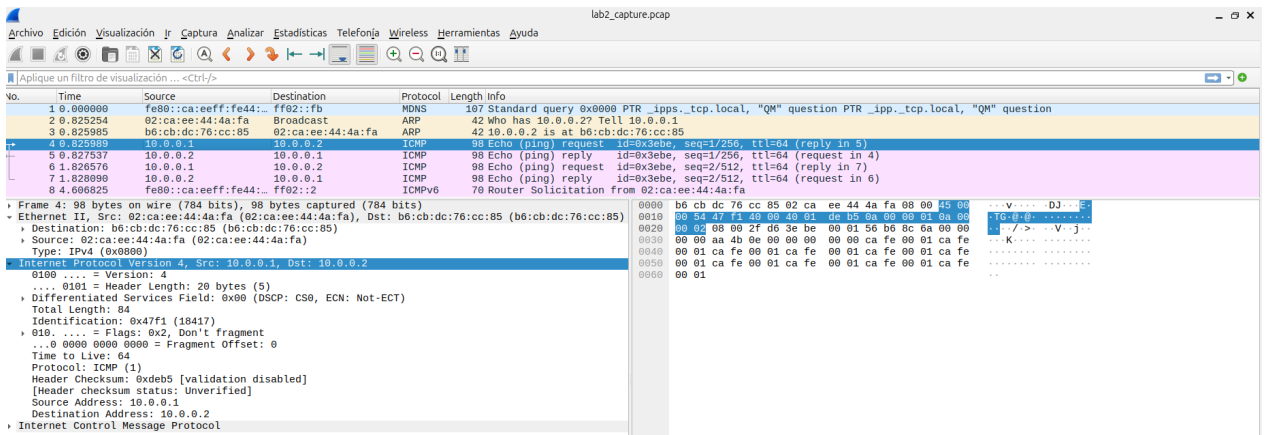


Figure 4: Lab 2: Wireshark capture; 98-byte *Echo Request* with EtherType 0x0800 and data ca fe 00 01.

Table 2 follows the *Echo Request* word by word on the transmit bus (figure 5, first two strips). Word 0 appears at $t = 351532$ ps and word 13 at $t = 351558$ ps, one per cycle.

Table 2: Lab 2: XGMII words of the *Echo Request* on the transmit bus.

Word	a_txc	a_txd	Content, in transmission order (lane 0 $\rightarrow$ 7)
idle	FF	0707070707070707	$8 \times /I/$ (idle)
0	01	D5555555555555FB	$/S/$ on lane 0, $6 \times$ preamble 55 and SFD D5
1	00	CA0285CC76DCCBB6	b6 cb dc 76 cc 85 (destination) and 02 ca (start of source)
2	00	00450008FA4A44EE	ee 44 4a fa (end of source), 08 00 (EtherType), 45 00 (IPv4)
3–12	00	(varies)	rest of IPv4/ICMP and data; e.g. FECA0100FECA0100 = 00 01 ca fe 00 01 ca fe
13	C0	07FDCF4531E30100	00 01 (last data), FCS e3 31 45 cf, $/T/$ (lane 6), $/I/$ (lane 7)
idle	FF	0707070707070707	back to idle

Observations:

- **Start.** TXC = 0x01 marks only lane 0 ($/S/$) as control; the preamble and SFD travel as data.
- **Terminate.** The 98-byte frame plus the 4-byte FCS adds up to 102 B: 12 full words and 6 bytes in the final word. That is why $/T/$ lands on lane 6 and TXC = 0xC0 (lanes 6 and 7 are control). The whole frame takes 14 cycles, versus 98 in lab 1: the bus is 8 times wider.

- **FCS.** It was checked offline that `e3 31 45 cf` matches the IEEE 802.3 CRC-32 computed over the 98 bytes of the captured frame (value `0xCF4531E3`, transmitted least significant byte first). The same holds for `7a 2b b1 ca` in the *Echo Reply*.
- **Reply.** On the receive bus (`a_rxd/a_rxc`, bottom two strips of figure 5) the *Reply* has the same structure, with destination and source swapped (`CBB6FA4A44EECA02`, ...).
- **Latency.** As in lab 1: one register per direction, with no visible offset between `a_txd` and `b_rxd` in the trace.

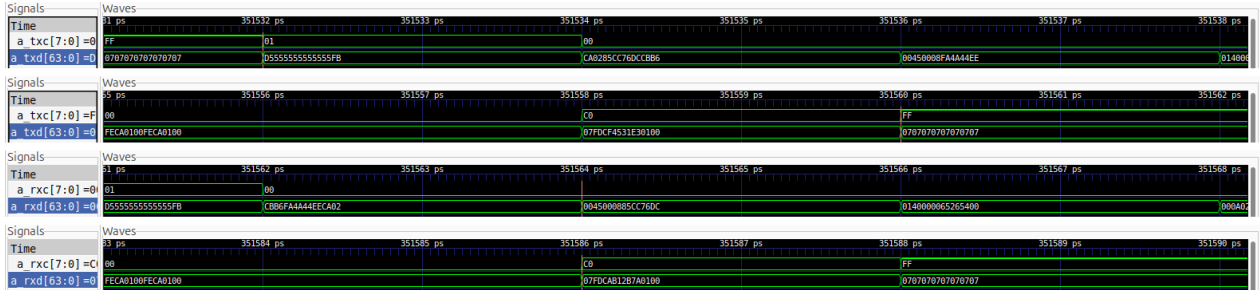


Figure 5: Lab 2: XGMII bus in GTKWave. From top to bottom: start of the *Request* (idle → start → header), end of the *Request* (data → FCS + /T/ → idle), start and end of the *Reply* on `a_rxd/a_rxc`.

Conclusion. XGMII introduces the 64-bit word with out-of-band control signaling (TXC). Frame delimitation no longer relies on a *valid* signal but on /S/ and /T/, and the FCS lets the receiver check integrity.

5 Lab 3: 64b/66b PCS and scrambler

5.1 Goal

Study the 10GBASE-R/100GBASE-R PCS (IEEE 802.3, clauses 49 and 82). Each XGMII word (64 data bits and 8 control bits) is encoded into a **66-bit block**: a 2-bit *sync header* and a 64-bit payload.

- **01: data block**, with 8 data octets.
- **10: control or mixed block.** Its first octet is the *Block Type Field* (BTF), which defines the format of the rest. Control characters are compressed: each /I/ becomes a 7-bit `0x00` code, and /S/ and /T/ become implicit in the BTF.

The overhead is 2 bits per 64 data bits, i.e. 3.125 % (versus 25 % for 8b/10b). The payload then goes through a **self-synchronizing scrambler** $G(x) = 1 + x^{39} + x^{58}$. The sync header is not scrambled.

5.2 Design

The topology is `host A <-> MAC A <-> CGMII <-> PCS A <=> (hdr + payload) <=> PCS B <-> MAC B <-> host B`. The MAC is now in the RTL: it builds the frame with preamble and SFD, pads it to 60 B, computes and appends the FCS, and generates /T/ and idles.

Two differences with the documentation shaped the analysis:

- Everything lives in a single `top.sv`, with signal names that differ from the documented ones. In the encoder (`top.pcs_a.encoder`), `raw_hdr` and `raw_payload` are the combinational block before the register and the scrambler, while `hdr` and `payload` are the registered output (scrambled when the scrambler is on). The decoder (`pcs_rx`) holds `descram_payload` and `block_type`.

- The `wrapper.cpp` of that version did not dump the trace, even though the build commands in the assignment assumed it did. FST dumping was added locally. The course fixed it in the repository later.

Two runs were made: phase 1 without the scrambler (the default) and phase 2 with `--enable-scrambler`.

5.3 Results: 64b/66b encoding

`ping -c 2 -p cafe0003` got 2 out of 2 replies. The capture (figure 6) has 8 packets: an ARP exchange, two *Echo* pairs and another ARP exchange started by `ns_b`. There is no IPv6 traffic anymore, because the updated network script disables it. One detail: the ARP generated by `ns_b` reaches `tap0` with **60 bytes**, while the one sent by the host is 42 bytes long. The RTL MAC TX pads frames to the 60-byte minimum and the MAC RX only strips the FCS, not the padding.

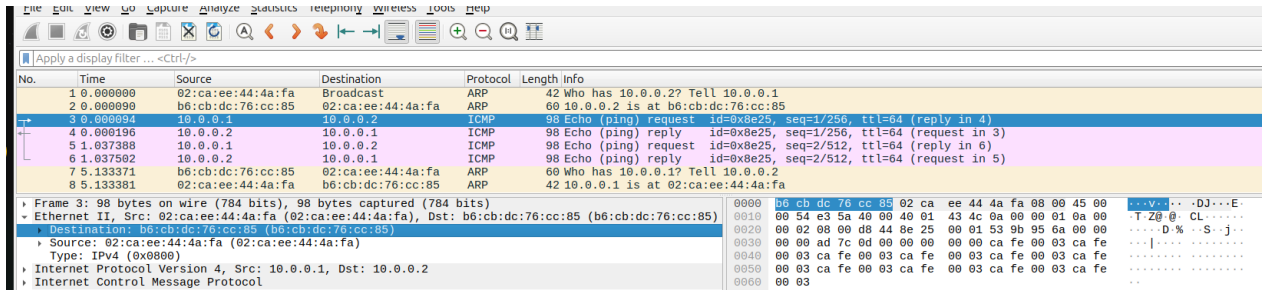


Figure 6: Lab 3: Wireshark capture (phase 1). The ARP coming from `ns_b` is 60 bytes long because of the padding added by the RTL MAC.

Table 3: Lab 3: blocks produced by the PCS A encoder (phase 1, scrambler off).

Block	cgmii_txc	cgmii_txd	raw_hdr	raw_payload
Idle	FF	0707070707070707	10	0000000000000001E
Start	01	D5555555555555FB	10	D555555555555578
Data	00	CA0285CC76DCCBB6	01	CA0285CC76DCCBB6
Terminate	C0	07FD15B839350300	10	0015B839350300E1

Table 3 and figure 7 show the four block types produced by the encoder:

- **Idle.** BTF `0x1E` (eight control characters). Each `/I/` is encoded as `0x00`, which is why the rest of the payload is zero.
- **Start.** BTF `0x78`. `/S/` becomes implicit and the remaining 7 bytes (preamble and SFD) travel as data.
- **Data.** Sync `01` and a payload equal to the XGMII word, with no BTF.
- **Terminate.** `/T/` on lane 6 $\rightarrow$ BTF `0xE1`. The 6 data octets are kept (`00 03` and the FCS `35 39 b8 15`); the `/I/` on lane 7 becomes `0x00`.

The registered outputs `hdr` and `payload` repeat these values one cycle later.

The closing BTF depends on the lane where `/T/` falls: `0x87`, `0x99`, `0xAA`, `0xB4`, `0xCC`, `0xD2`, `0xE1` and `0xFF` for lanes 0 to 7. Scanning the whole phase-1 trace, each encoder produced 4 *start* blocks and 4 *terminate* blocks:

- **2 with 0xE1:** the ICMP frames ($98 + 4 = 102$ B) leave 6 bytes in the last word.
- **2 with 0x87:** the ARP frames, padded to 60 B, plus 4 bytes of FCS add up to 64 B, exactly 8 words. `/T/` therefore falls on lane 0 of a new word.

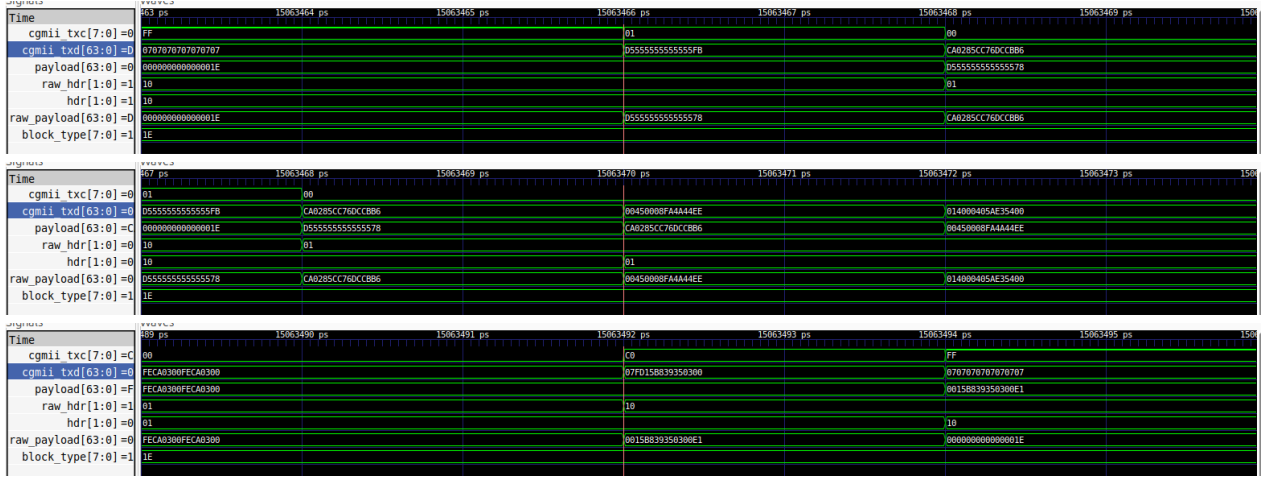


Figure 7: Lab 3, phase 1: encoding in GTKWave. Top: idle → start (BTF 0x78). Middle: start → data blocks (*raw_hdr* = 01). Bottom: data → terminate (BTF 0xE1) → idle.

5.4 Results: scrambler

The Wireshark capture is identical with and without the scrambler: scrambling is fully undone in PCS B, before reaching the MAC and the TAP. Only the trace reveals it (figure 8):

- Idle blocks, which are always ...001E without the scrambler, now look pseudo-random (6204FEC71D2BD3BD, 7C6E7CCA1E0BCCDE, ...). That is the purpose of the scrambler: long runs of idles or repetitive data would provide few transitions for clock recovery and a spectrum with strong lines and DC imbalance.
- *hdr* does not change (10/01). The sync header is never scrambled because the receiver needs it to find block boundaries (*block lock*) before descrambling.
- In PCS B, *descram_payload* reproduces the *raw_payload* of PCS A one cycle later, the delay of the encoder output register (table 4).

Table 4: Lab 3, phase 2: payload before and after the scrambler, and its recovery at the receiver.

t (ps)	raw_payload (PCS A)	payload one cycle later	descram_payload (PCS B)
10294160	00006A95BBB90100	CAA83350E5586F99	00006A95BBB90100
10294162	0000000B82B30000	ACCE46A3953FC9CC	0000000B82B30000
10294164	FECA0300FECA0000	96AE4CCC17BD AEED	FECA0300FECA0000
10294166	FECA0300FECA0300	E5D9627899B293B0	FECA0300FECA0300

The last two rows show that **the same input word produces different outputs**, because the output depends on the history of the stream. It was also checked offline, using the equation $s_i = d_i \oplus s_{i-39} \oplus s_{i-58}$, that each *payload* is exactly the scrambled version of the previous cycle's *raw_payload*, and that applying the descrambler returns the original value.

Since the descrambler builds its state from the same scrambled bits it receives, no seed has to be shared: it synchronizes by itself after 58 bits. The trade-off is that a single bit error on the line is multiplied by 3 at the output (positions i , $i + 39$ and $i + 58$).

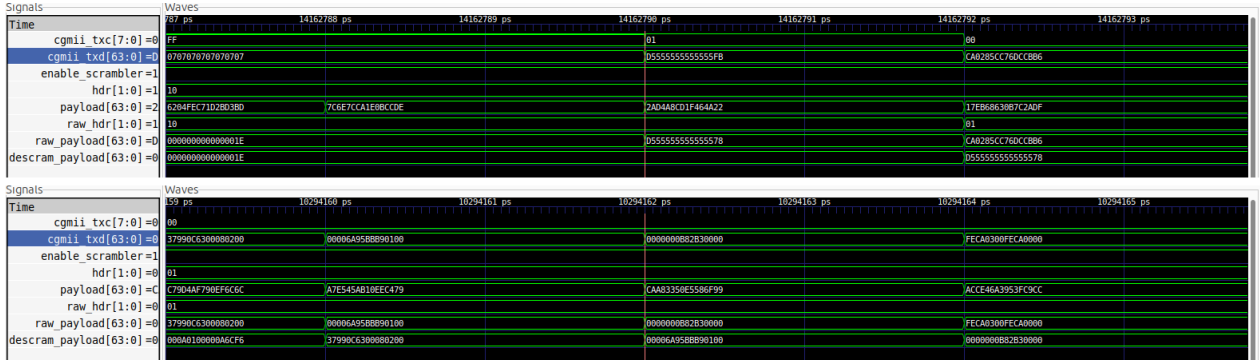


Figure 8: Lab 3, phase 2 (scrambler on): top, scrambled idle blocks and the transition to start; bottom, data blocks (raw_payload, payload and descram_payload).

Conclusion. The PCS completely changes how data is represented on the link (scrambled 66-bit blocks), yet it is transparent to the upper layers: Wireshark sees exactly the same frames.

6 Lab 4: 100GBASE-R PCS and Multi-Lane Distribution (MLD)

6.1 Goal

Understand how 100GBASE-R spreads the stream of 66-bit blocks over **20 virtual lanes (PCS lanes)** and periodically inserts **Alignment Markers (AM)** so that the receiver can identify each lane, compensate skew and reorder the lanes. The assignment asks to verify the round-robin distribution, the AM periodicity and format, and the propagation of the sideband signals from transmitter to receiver.

6.2 Real mechanism (clause 82) and the lab abstraction

In a real 100GBASE-R PCS, blocks are distributed round-robin over 20 lanes. Every 16383 blocks per lane, an AM is inserted on **all** lanes at once; the PCS makes room for it by deleting idles. Each AM has 8 octets, M0 M1 M2 BIP3 M4 M5 M6 BIP7, where M4..M6 are the complement of M0..M2 and BIP7 is the complement of BIP3. For example, lane 0 uses C1 68 21 and 3E 97 DE. BIP3 is a bit-interleaved parity computed over the lane since the previous AM. The receiver locks each lane, identifies it by its AM, aligns the lanes (*deskew*) and reorders them.

The lab simplifies this mechanism so that it can be followed in GTKWave (table 5).

Table 5: Lab 4: real MLD versus the lab abstraction.

Aspect	Clause 82	Lab RTL
Distribution	20 lanes (actual demultiplexing)	a single block stream, never split
AM interval	16383 blocks per lane	AM_INTERVAL = 64 cycles (parameter)
AM insertion	replaces blocks (compensated with idles)	sideband signal: the data block is never replaced
Lane identification	implicit in the 8-octet pattern	explicit: mld_lane_id and a byte inside mld_am_pattern
BIP3/BIP7	real parity, detects errors	not implemented (fixed filler 00 A5 A5 A5 A5 A5)
Deskew and reordering	per-lane FIFOs at the receiver	not needed: no skew by construction

The core of the MLD transmitter (mld_tx) is a lane counter and a block counter, both registered in the same always_ff as the output block (condensed version of the code):

```

mld_lane_id    <= lane_ptr;                                // lane of the current block
lane_ptr       <= (lane_ptr == NUM_LANES-1) ? 0 : lane_ptr + 1;
mld_am_pattern <= {2'b10, 8'hC1, 8'(lane_ptr), 48'h00_A5_A5_A5_A5};
mld_is_am      <= (block_cnt == AM_INTERVAL-1);           // pulse every AM_INTERVAL
block_cnt      <= (block_cnt == AM_INTERVAL-1) ? 0 : block_cnt + 1;
pcs_block_out  <= pcs_block_in;                            // (or scrambled)

```

6.3 Execution

The emulator was built with `-GAM_INTERVAL=64 -GNUM_LANES=20`; in this version the trace is always generated. `ping -c 5 -p feca0004` got 5 out of 5 replies (0 % loss) and `tcpdump` captured 12 packets: ARP and five *Echo* pairs (figure 9).

With the lane order described in section 2, the ICMP data `fe ca 00 04 ...` shows up on the bus as `1CAFE0400CAFE0400` blocks (sync 01 followed by the payload). In this version:

- The encoder places the BTF in bits [63:56]. For example, an idle block reads `21E0000000000000` (sync 10 followed by 1E), unlike lab 3, where the BTF sits in [7:0].
- The RTL MAC is simplified: it adds neither FCS nor padding. That is why the ARP reply arrives with 42 B, and why the 98-byte ICMP frame closes with BTF `0xAA` (/T/ on lane 2) instead of `0xE1`.

```

+ lab4-pcs-mld sudo ./obj_dir/emulator
[TAP] Bound successfully to interface: tap0
[TAP] Bound successfully to interface: tap1
=====
[EMULATOR] Lab 4 Dual TAP Node A (tap0) <-> Node B (ns_b/tap1) Active
[EMULATOR] Waveform Dumping: ACTIVE (dump.fst)
[EMULATOR] Press Ctrl+C to stop simulation cleanly.
=====
^C
[EMULATOR] Flushing waveform buffers...
[EMULATOR] Saved waveform to dump.fst successfully.
+ lab4-pcs-mld

+ sudo tcpdump -i tap0 -w ~/Documentos/protocolos/TP3/capturas/lab4/vcd_pcap_originales/lab4_cafe0004.pcapng
[sudo] password for francisco-javier-vasquez:
tcpdump: listening on tap0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C12 packets captured
12 packets received by filter
0 packets dropped by kernel
+ -

```

```

+ ping -c 5 -p feca0004 -I tap0 10.0.0.2
PATTERN: 0xfeca0004
PING 10.0.0.2 (10.0.0.2) from 10.0.0.1 tap0: 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=215 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.078 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.083 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=31.4 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.071 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4072ms
rtt min/avg/max/mdev = 0.071/49.482/215.396/83.877 ms
+ -

```

Figure 9: Lab 4: emulator, `ping -c 5 -p feca0004` (5/5, 0 % loss) and `tcpdump` (12 packets).

6.4 Answers to the assignment

Task 1 (questions 1.1 and 1.2). `mld_lane_id` increments from 0 to 19 and wraps back to 0 immediately (figure 10). One block is dispatched on every clock edge: `pcs_block_out` is updated every cycle, together with `mld_lane_id`, with no gaps or stalls. In the figure, the blocks of an ICMP frame (`278D55...` for *start*, `1CAFE0400CAFE0400` for data, `2AA000...` for *terminate*) go by while the lane counter keeps running.

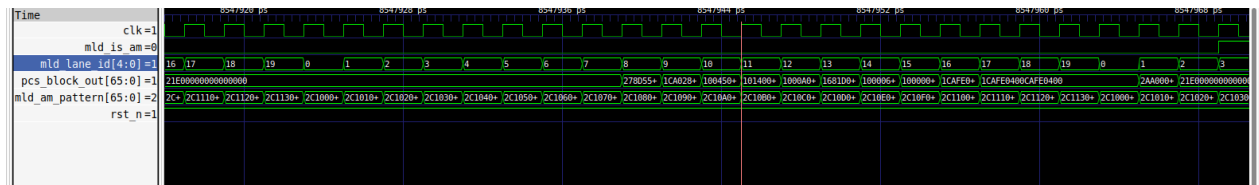


Figure 10: Lab 4, task 1: `mld_lane_id` (in decimal) cycles through 0 to 19 while `pcs_block_out` carries an ICMP frame.

Task 2 (questions 2.1 to 2.3).

- **2.1.** `mld_is_am` pulses every 64 cycles: 128 ps between pulses, for example at 129 ps and at 257 ps (figure 11). This matches `-GAM_INTERVAL=64`. Over the full trace, the pulses continue at 385, 513, 641, 769 ps, and so on.
- **2.2.** Lanes 0 and 5 **never** coincide with an AM pulse. Pulse k occurs at cycle $64k$, when the lane is $(64k - 1) \bmod 20 = (4k - 1) \bmod 20$. Since $\gcd(64, 20) = 4$, only lanes **3, 7, 11, 15 and 19** appear (repeating every 5 pulses, i.e. every 320 cycles), which was confirmed on the trace. The captured values are `VL3 = 2C10300A5A5A5A5A5` and `VL7 = 2C10700A5A5A5A5A5`. By formula, `VL0 = 2C10000A5A5A5A5A5` and `VL5 = 2C10500A5A5A5A5A5`.
- **2.3.** The field that changes with the lane is byte `[55:48]` of `mld_am_pattern`, the one right after `C1`: it holds the lane number (03, 07, ...). The rest of the pattern is fixed: `{10, C1, lane, 00, A5 A5 A5 A5 A5}`.

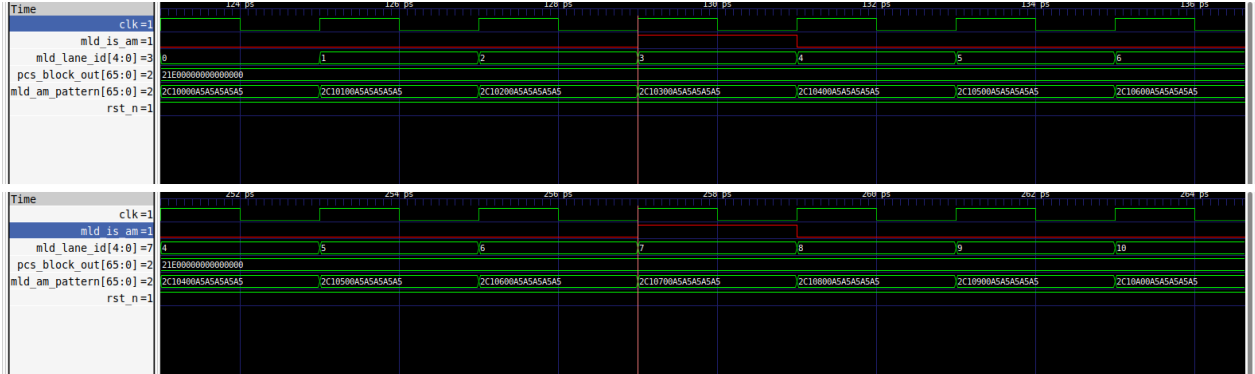


Figure 11: Lab 4, task 2: `mld_is_am` pulses at 129 ps (lane 3, top) and at 257 ps (lane 7, bottom), 64 cycles apart; the lane byte in `mld_am_pattern` follows `mld_lane_id`.

Task 3 (questions 3.1 and 3.2).

- **3.1.** The delay is **0 cycles**. In `top.sv`, `mld_lane_id_a2b`, `mld_is_am_a2b` and `mld_am_pattern_a2b` connect the outputs of `mld_tx_inst_a` directly to the inputs of `mld_rx_inst_b`, with no registers in between. In the trace, both hierarchical names share the same identifier: they are the same net.
- **3.2.** Yes. `pcs_block_out` and `mld_lane_id` are updated in the same `always_ff` of `mld_tx`, so each block reaches `mld_rx` together with its lane label. `mld_rx` does not use the sideband: it only descrambles (when enabled) and registers the block, which adds one cycle to the data path.

Conclusion. The lab shows the timing mechanism of MLD (round-robin labeling and periodic markers) without the complexity of deskew. The choice of parameters has an undocumented consequence: with `AM_INTERVAL` a multiple of 4 and 20 lanes, only 5 lanes ever “see” an AM. This does not happen in the standard, because AMs are inserted on all lanes simultaneously.

7 General conclusions

- **Each sublayer changes the representation on the link to solve a physical problem.** Data goes from bytes (lab 1) to 64-bit words with out-of-band control (lab 2), then to scrambled 66-bit blocks (lab 3), and finally to 20 virtual lanes with alignment markers (lab 4). The problems solved are, in that order: bus width versus clock frequency, frame delimitation, transition density and DC balance, and distribution over parallel lanes with skew.
- **The sublayers are transparent.** In all four labs `ping` worked with no loss and Wireshark showed the same frames: same MAC addresses, EtherType and payload. The differences only appear in the RTL trace. That is why the core method of the assignment was correlating the Wireshark hex dump with the cycle-by-cycle values in GTKWave, taking the lane order into account.

- **Reading the RTL was essential.** The documented signal names did not match the real ones, lab 2 changed version during the course and lab 3 did not dump its trace. In every case, interpreting the waveforms required starting from the actual code.
- **Bus width and cycles.** The same 98-byte frame takes 98 cycles on the 8-bit bus and 14 on the 64-bit bus. This even shows in the `ping` RTT: 10–15 ms in lab 1 versus 1.5–2.3 ms in lab 2, a ratio close to $98/14 = 7$. Both wrappers pause (`usleep`) on every simulated cycle, so the RTT measures the speed of the simulation rather than that of a physical link. In labs 3 and 4, without that pause, the RTT typically drops below 0.25 ms.